

Implémentation d'un Système Multi-Agents Décentralisé pour la Gestion d'Énergie sur Blockchain Ethereum

Résumé

Ce rapport présente une implémentation novatrice d'un système multi-agents (MAS) pour la gestion d'énergie, basé sur la technologie blockchain Ethereum. Le système proposé offre une architecture décentralisée permettant l'interaction autonome entre différents agents énergétiques (bâtiments, véhicules électriques, stockage, production photovoltaïque et réseau électrique), optimisant l'allocation des ressources et garantissant la transparence des transactions énergétiques. L'approche décentralisée, utilisant des contrats intelligents sur la blockchain Ethereum, élimine le besoin d'un intermédiaire central tout en offrant un historique immuable des échanges énergétiques. Les résultats des tests démontrent la faisabilité et l'efficacité de cette approche pour la gestion décentralisée de l'énergie dans le contexte des smart grids et des communautés énergétiques intelligentes.

1. Introduction

La transition énergétique mondiale entraîne une décentralisation croissante des systèmes énergétiques, avec l'intégration massive d'énergies renouvelables intermittentes et de nouveaux acteurs comme les prosommateurs et les véhicules électriques. Cette décentralisation pose de nouveaux défis en termes de coordination, de sécurité et de transparence des échanges énergétiques. Les approches traditionnelles, basées sur des systèmes centralisés, présentent plusieurs limitations: points uniques de défaillance, vulnérabilité aux cyberattaques, problèmes de confidentialité et optimisation sous-optimale des ressources.

La technologie blockchain, avec sa nature distribuée et son immuabilité, apparaît comme une solution prometteuse pour répondre à ces défis. En particulier, la blockchain Ethereum, avec sa capacité à exécuter des contrats intelligents (smart contracts), offre des fonctionnalités avancées pour l'implémentation de systèmes décentralisés complexes.

Dans ce contexte, notre étude propose une architecture novatrice intégrant un système multi-agents (MAS) à la blockchain Ethereum pour la gestion décentralisée de l'énergie. Cette approche permet:

- L'automatisation des échanges énergétiques via des contrats intelligents
- Une allocation optimale des ressources basée sur les préférences des agents
- La traçabilité et la transparence des transactions
- L'élimination des intermédiaires centralisés

Ce rapport détaille l'architecture du système, les mécanismes d'interaction entre agents, l'implémentation des contrats intelligents, et présente les résultats des simulations réalisées.

2. État de l'art et cadre théorique

2.1 Systèmes multi-agents pour la gestion d'énergie

Les systèmes multi-agents (MAS) constituent un paradigme puissant pour modéliser des systèmes complexes composés d'entités autonomes interagissant entre elles. Dans le domaine énergétique, les MAS ont été largement utilisés pour simuler et optimiser les interactions entre différents acteurs du réseau (Coelho et al., 2017; Ramchurn et al., 2012).

L'approche MAS présente plusieurs avantages pour la gestion décentralisée de l'énergie:

- Modélisation naturelle des différents acteurs du réseau en tant qu'agents autonomes
- Prise en compte des préférences individuelles et des contraintes locales
- Mécanismes de négociation et d'enchères pour l'allocation optimale des ressources
- Adaptabilité aux changements dynamiques du système

Cependant, les implémentations traditionnelles de MAS reposent souvent sur des plateformes centralisées ou des systèmes de communication qui ne garantissent pas la transparence et l'immuabilité des transactions.

2.2 Blockchain et contrats intelligents pour l'énergie

La blockchain est une technologie de registre distribué permettant le stockage d'informations de manière sécurisée, transparente et immuable sur un réseau de nœuds. Ethereum, en particulier, est une blockchain programmable permettant l'exécution de contrats intelligents - des programmes auto-exécutables dont les conditions sont directement inscrites dans le code (Buterin, 2014).

Dans le secteur énergétique, la blockchain a été proposée pour diverses applications:

- Marchés d'énergie pair-à-pair (Mengelkamp et al., 2018)
- Gestion des certificats d'énergie renouvelable (Andoni et al., 2019)
- Suivi et traçabilité de la provenance de l'énergie (Mihaylov et al., 2014)
- Facturation et paiement automatisés (Pop et al., 2018)

Les contrats intelligents, en particulier, offrent un cadre idéal pour automatiser les échanges énergétiques selon des règles prédéfinies, sans nécessiter d'intermédiaire de confiance.

2.3 Lacunes dans la recherche actuelle

Malgré les avancées dans ces deux domaines, l'intégration complète des systèmes multi-agents avec la blockchain pour la gestion décentralisée de l'énergie reste un domaine de recherche émergent. Les études existantes se concentrent souvent sur des cas d'usage spécifiques (comme les marchés d'énergie pair-à-pair) sans proposer d'architecture holistique intégrant différents types d'agents énergétiques.

Par ailleurs, les questions d'évolutivité, de consommation énergétique des protocoles de consensus et d'interaction entre la simulation MAS et la blockchain en temps réel sont souvent négligées.

Notre étude vise à combler ces lacunes en proposant une architecture complète pour l'intégration MAS-blockchain dans le contexte énergétique.

3. Architecture du système

3.1 Vue d'ensemble

L'architecture proposée comprend deux composantes principales interconnectées:

- 1. Un système multi-agents (MAS) implémentant la logique des différents agents énergétiques
- 2. Une infrastructure blockchain basée sur Ethereum pour l'enregistrement des transactions et l'exécution des contrats intelligents

La Figure 1 présente une vue d'ensemble de l'architecture du système.

 Copier

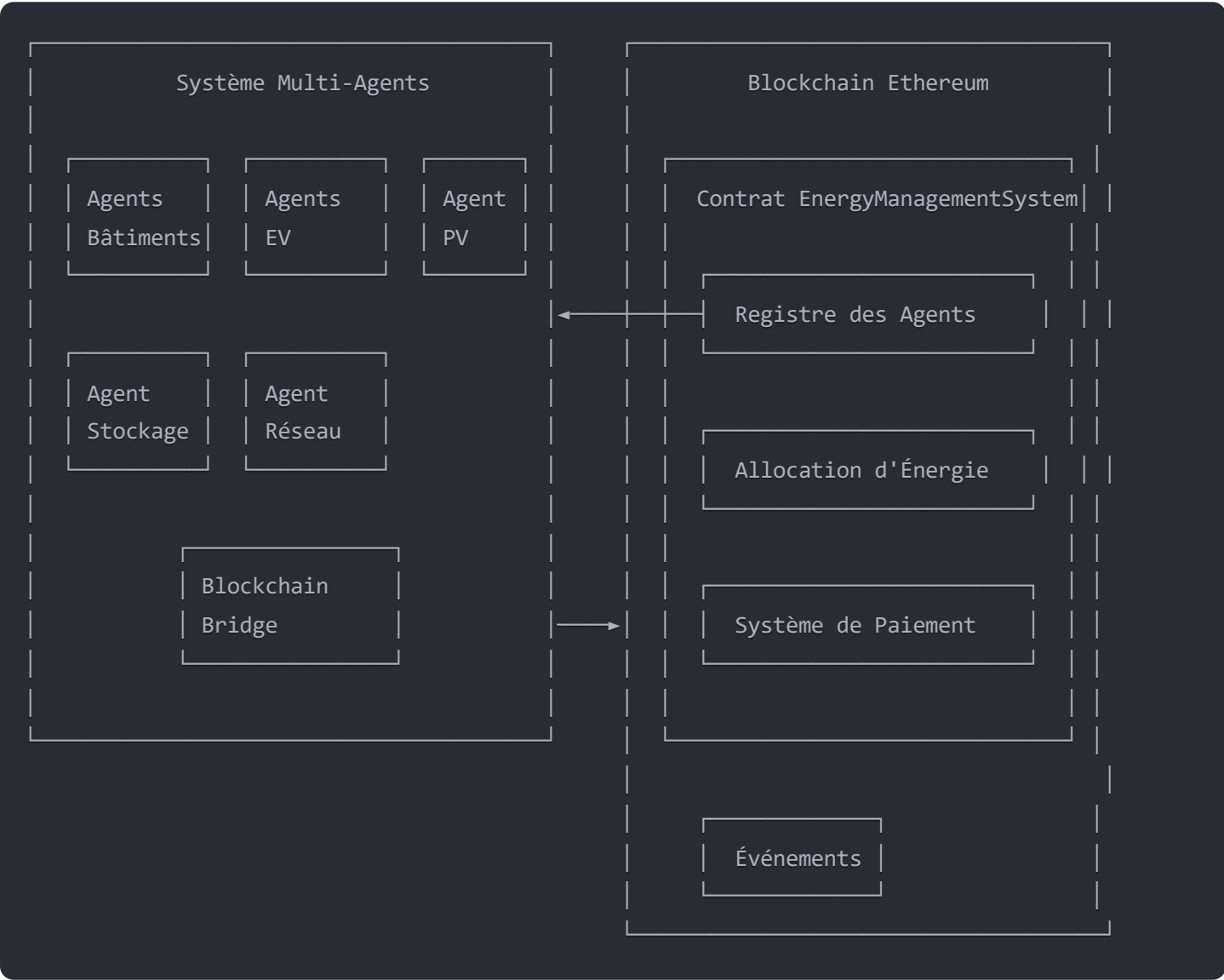


Figure 1: Architecture du système MAS-Blockchain

3.2 Types d'agents et leurs caractéristiques

Notre système comprend cinq types d'agents énergétiques:

1. **Agents Bâtiments:** Représentent les consommateurs d'énergie avec des profils de demande spécifiques. Chaque bâtiment a des préférences concernant ses sources d'approvisionnement (réseau, PV, stockage).
2. **Agents Véhicules Électriques (EV):** Représentent des charges mobiles avec des contraintes temporelles (périodes d'arrivée/départ) et des besoins de charge spécifiques (état de charge initial, état de charge requis).
3. **Agent Stockage:** Gère un système de stockage d'énergie avec des contraintes techniques (capacité, puissance de charge/décharge, efficacité) et des objectifs économiques.
4. **Agent PV:** Représente une source de production photovoltaïque avec un profil de génération variable.
5. **Agent Réseau:** Représente le réseau électrique conventionnel, fournissant l'énergie non couverte par les sources renouvelables locales.

Chaque agent possède:

- Des attributs physiques et techniques
- Des préférences exprimées par des coefficients (alpha, beta, gamma) pour les différentes sources d'énergie
- Une fonction objectif prenant en compte le coût, le confort et l'utilisation des énergies renouvelables

3.3 Contrats intelligents

Le cœur du système blockchain est le contrat intelligent `EnergyManagementSystem.sol` qui implémente plusieurs fonctionnalités:

1. **Registre des agents:** Enregistre et maintient les informations sur tous les agents du système, leurs préférences et leurs caractéristiques techniques.
2. **Système de tarification:** Gère les coûts variables de l'énergie (PV, réseau, stockage) selon l'heure de la journée.
3. **Mécanisme d'allocation d'énergie:** Implémente les algorithmes d'allocation d'énergie entre les agents selon leurs préférences.
4. **Système de paiement:** Gère les transactions financières entre les agents.

Le contrat utilise plusieurs structures de données sophistiquées:

- Enums pour les types d'agents
- Structs pour les données des agents, les préférences et les coûts
- Mappings pour l'accès efficace aux informations des agents

3.4 Interface blockchain (Bridge)

L'interface entre le système multi-agents et la blockchain est assurée par un composant

`blockchain_bridge.py` qui:

- Établit la connexion avec la blockchain Ethereum (Ganache dans notre implémentation)
- Convertit les données du format MAS au format blockchain
- Gère les transactions et les appels aux fonctions du contrat intelligent
- Récupère les résultats et événements de la blockchain

Cette interface joue un rôle crucial en permettant la communication bidirectionnelle entre la simulation MAS et l'infrastructure blockchain.

4. Mécanismes d'interaction et consensus

4.1 Protocole de consensus Ethereum

Notre implémentation repose sur l'infrastructure Ethereum, qui utilise actuellement le mécanisme de consensus Proof of Work (PoW) sur le réseau principal, avec une transition progressive vers Proof of Stake (PoS) via Ethereum 2.0. Dans notre environnement de test utilisant Ganache, nous utilisons un consensus simplifié adapté aux environnements de développement, basé sur un mécanisme d'autorité unique.

Le protocole de consensus garantit:

- La validation des transactions par le réseau
- L'ordonnancement déterministe des transactions
- La protection contre les doubles dépenses et autres attaques
- La finalité des transactions après un certain nombre de confirmations

4.2 Cycle de vie d'une transaction énergétique

Le cycle de vie complet d'une transaction énergétique dans notre système comprend plusieurs étapes:

1. **Initialisation:** Les agents s'enregistrent sur la blockchain avec leurs caractéristiques et préférences.
2. **Mise à jour des ressources:** À chaque pas de temps (heure), l'agent PV et l'agent Stockage mettent à jour leurs ressources disponibles.
3. **Demande d'allocation:** Les agents consommateurs (Bâtiments, EVs) demandent l'allocation d'énergie selon leurs besoins.
4. **Exécution du contrat:** Le contrat intelligent alloue l'énergie en fonction des préférences des agents et des ressources disponibles.
5. **Enregistrement des coûts:** Les coûts sont calculés et enregistrés pour chaque agent.
6. **Paiement:** À la fin de la simulation, les paiements sont effectués entre les agents.

Ce cycle se répète à chaque pas de temps, créant un flux continu de transactions sur la blockchain.

4.3 Événements et traçabilité

Le contrat intelligent émet plusieurs types d'événements qui sont enregistrés sur la blockchain:

- `AgentRegistered`: Émis lors de l'enregistrement d'un nouvel agent
- `EnergyAllocated`: Émis lors de l'allocation d'énergie entre agents
- `HourAdvanced`: Émis lorsque l'heure de simulation avance
- `PriceUpdated`: Émis lors de la mise à jour des prix de l'énergie
- `EnergyCostCalculated`: Émis après le calcul du coût total pour un agent
- `PaymentProcessed`: Émis lors du traitement d'un paiement
- `ResourcesUpdated`: Émis lors de la mise à jour des ressources disponibles

Ces événements permettent une traçabilité complète des interactions et transactions, offrant une transparence totale sur le fonctionnement du système.

4.4 Mécanismes d'allocation d'énergie

L'allocation d'énergie est réalisée par le contrat intelligent selon les préférences des agents:

1. **Pour les bâtiments** (fonction `allocateBuildingEnergy`):
 - L'énergie PV est allouée selon le coefficient beta
 - L'énergie de stockage est allouée selon le coefficient gamma
 - L'énergie du réseau est allouée selon le coefficient alpha
2. **Pour les véhicules électriques** (fonction `allocateEVEnergy`):
 - Allocation similaire, mais uniquement pendant les périodes d'arrivée/départ
 - Prise en compte de l'état de charge actuel et requis

Les préférences des agents (alpha, beta, gamma) jouent un rôle crucial dans cette allocation, permettant d'exprimer des stratégies comme la priorité aux énergies renouvelables ou la minimisation des coûts.

5. Implémentation technique

5.1 Environnement de développement

L'implémentation technique du système repose sur les technologies suivantes:

- **Python 3.8+** pour la simulation multi-agents, utilisant:
 - Mesa (framework multi-agents)
 - NumPy et Pandas pour le traitement des données
 - Web3.py pour l'interaction avec la blockchain
- **Solidity 0.8.19** pour les contrats intelligents

- **Truffle** comme framework de développement, test et déploiement des contrats intelligents
- **Ganache** comme blockchain Ethereum locale pour le développement et les tests

5.2 Structure du code

Le projet est organisé selon la structure suivante:

 Copier

```
projet/
|-- MAS.py (simulation multi-agents)
|-- blockchain_energy/
|   |-- contracts/
|   |   |-- EnergyManagementSystem.sol (contrat principal)
|   |   |-- Migrations.sol
|   |-- migrations/
|   |   |-- 1_initial_migration.js
|   |   |-- 2_deploy_contracts.js
|   |-- src/
|   |   |-- blockchain_bridge.py (pont entre Python et blockchain)
|   |   |-- deploy.js (script de déploiement)
|-- truffle-config.js (configuration Truffle)
|-- package.json (dépendances npm)
```

5.3 Contrat intelligent EnergyManagementSystem

Le contrat intelligent `EnergyManagementSystem.sol` est écrit en Solidity et comprend plus de 500 lignes de code. Ses principales fonctionnalités sont:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract EnergyManagementSystem {
    // Structures de données et variables d'état

    // Fonctions d'enregistrement des agents
    function registerPVAgent(...) public onlyOwner returns (uint256) {...}
    function registerStorageAgent(...) public onlyOwner returns (uint256) {...}
    function registerGridAgent(...) public onlyOwner returns (uint256) {...}
    function registerBuildingAgent(...) public onlyOwner returns (uint256) {...}
    function registerEVAgent(...) public onlyOwner returns (uint256) {...}

    // Fonctions d'allocation d'énergie
    function updateEnergySources(...) public {...}
    function allocateBuildingEnergy(uint256 buildingId) public {...}
    function allocateEVEnergy(uint256 evId) public {...}

    // Fonctions de gestion des coûts et paiements
    function calculateTotalCost(uint256 agentId) public view returns (uint256) {...}
    function processPayment(...) public onlyOwner {...}

    // Fonctions utilitaires
    function advanceHour() public onlyOwner {...}
    function updatePrices(...) public onlyOwner {...}
}
```

5.4 Interface blockchain (Bridge)

L'interface blockchain `blockchain_bridge.py` assure la communication entre la simulation MAS et la blockchain:

```
class BlockchainBridge:
    def __init__(self, ganache_url="http://127.0.0.1:7545"):
        # Initialisation de la connexion blockchain

    def connect_to_contract(self, contract_address):
        # Connexion au contrat intelligent

    def get_agent_count(self):
        # Récupération du nombre d'agents

    def process_payment(self, agent_id, account_index):
        # Traitement des paiements
```

5.5 Gestion des identifiants blockchain

Un aspect critique de l'implémentation est la gestion des identifiants (IDs) des agents entre le système MAS et la blockchain. Dans notre implémentation, chaque agent possède:

- Un `unique_id` local dans la simulation MAS
- Un `blockchain_id` correspondant à son identifiant sur la blockchain

La correspondance entre ces deux IDs est maintenue tout au long de la simulation, permettant une communication cohérente entre les deux systèmes.

6. Résultats et analyse

6.1 Scénario de test

Pour valider notre système, nous avons réalisé des simulations sur 24 heures avec les agents suivants:

- 4 bâtiments avec des profils de consommation différents
- 4 véhicules électriques avec différentes caractéristiques et périodes de charge
- 1 agent de stockage
- 1 agent PV avec un profil de production variable
- 1 agent réseau

Les données de consommation et de production sont basées sur des profils réels, avec une granularité horaire.

6.2 Résultats économiques

Les résultats économiques de la simulation montrent:

Agent	Coût (\$)	Comfort	PV Utilisé (kWh)
PV	35.53	N/A	355.27
Storage	-2.16	0.69	6.24
Building 1	-57.60	1.00	99.48
Building 2	-108.05	1.00	89.53
Building 3	-208.52	1.00	80.58
Building 4	-44.76	1.00	72.52
EV 1	-1.08	1.00	2.77
EV 2	-0.95	1.00	1.14
EV 3	-0.37	1.00	2.00
EV 4	-1.60	1.00	1.03

Observations clés:

- L'agent PV génère un revenu positif de 35.53\$
- Tous les consommateurs atteignent un niveau de confort optimal (1.0)
- L'agent stockage présente un confort de 0.69, reflétant son rôle tampon
- Le confort moyen global est de 0.97
- Le coût total du système est de -389.58\$

Les résultats blockchain montrent les coûts enregistrés pour chaque agent:

Agent	Coût Blockchain (ETH)
PV	0
Storage	0
Grid	0
Building 1	4.81
Building 2	8.83
Building 3	16.85
Building 4	3.73
EV 1	~0
EV 2	~0
EV 3	~0
EV 4	~0

6.3 Analyse des transactions blockchain

L'analyse des transactions blockchain révèle plusieurs informations intéressantes:

1. **Volume de transactions:** La simulation génère environ 240 transactions sur 24 heures, correspondant aux allocations d'énergie et aux avancées temporelles.
2. **Consommation de gaz:** Les opérations les plus coûteuses en termes de gaz sont:
 - L'enregistrement des agents (4.2 millions de gaz pour l'agent PV)
 - L'allocation d'énergie pour les grands bâtiments (~100 000 gaz par allocation)
3. **Temps de confirmation:** Dans notre environnement Ganache, les transactions sont confirmées presque instantanément, mais ce délai serait plus important sur le réseau Ethereum principal.
4. **Évènements:** Plus de 300 événements sont émis au cours de la simulation, fournissant une traçabilité complète des interactions.

6.4 Limites et considérations d'évolutivité

Notre implémentation présente certaines limitations:

1. **Évolutivité:** Le nombre d'agents et de transactions supportés est limité par les contraintes de la blockchain Ethereum (limite de gaz par bloc).
2. **Coûts de transaction:** Sur le réseau Ethereum principal, les coûts de transaction (gas fees) pourraient rendre certaines opérations économiquement non viables.
3. **Latence:** Les délais de confirmation des transactions pourraient affecter les applications en temps réel.
4. **Consommation énergétique:** L'utilisation du consensus PoW d'Ethereum génère une empreinte environnementale significative.

Pour adresser ces limitations, plusieurs solutions pourraient être envisagées:

- Utilisation de solutions de couche 2 (Layer 2) comme les sidechains ou rollups
- Migration vers des blockchains plus efficaces énergétiquement (PoS)
- Optimisation des contrats pour réduire la consommation de gaz

7. Conclusion et perspectives

7.1 Synthèse des contributions

Cette étude a proposé et validé une architecture novatrice intégrant un système multi-agents avec la blockchain Ethereum pour la gestion décentralisée de l'énergie. Les principales contributions sont:

1. Une architecture complète pour l'interaction entre MAS et blockchain dans le contexte énergétique
2. Un contrat intelligent sophistiqué gérant l'enregistrement des agents, l'allocation d'énergie et les paiements
3. Une interface blockchain-MAS permettant une communication bidirectionnelle entre les deux systèmes

4. Une validation expérimentale démontrant la faisabilité et l'efficacité de l'approche

Les résultats montrent que cette approche permet une allocation efficace des ressources énergétiques tout en garantissant la transparence et la traçabilité des transactions.

7.2 Implications pour les communautés énergétiques

Cette recherche ouvre des perspectives prometteuses pour les communautés énergétiques décentralisées:

- **Marchés locaux d'énergie:** L'architecture proposée pourrait servir de base pour l'implémentation de marchés locaux d'énergie pair-à-pair.
- **Micro-réseaux intelligents:** Le système pourrait être adapté pour la gestion autonome de micro-réseaux, optimisant l'équilibre production-consommation.
- **Flexibilité énergétique:** Les mécanismes d'allocation basés sur les préférences pourraient faciliter la valorisation de la flexibilité des consommateurs.
- **Traçabilité de l'énergie verte:** La blockchain garantit la provenance et la traçabilité de l'énergie renouvelable.

7.3 Perspectives de recherche future

Plusieurs axes de recherche peuvent être explorés pour étendre ce travail:

1. **Optimisation multi-objectifs:** Intégrer des algorithmes d'optimisation plus sophistiqués prenant en compte des objectifs environnementaux, économiques et techniques.
2. **Mécanismes d'enchères:** Développer des mécanismes d'enchères pour l'allocation dynamique des ressources énergétiques.
3. **Solutions d'évolutivité blockchain:** Explorer l'utilisation de solutions de couche 2 ou de blockchains alternatives pour améliorer l'évolutivité et réduire les coûts.
4. **Intégration IoT:** Connecter le système à des dispositifs IoT pour l'acquisition de données en temps réel.
5. **Validation à grande échelle:** Tester le système avec un grand nombre d'agents et dans des conditions réelles.

Cette recherche constitue une étape importante vers la réalisation de systèmes énergétiques véritablement décentralisés, autonomes et transparents, contribuant ainsi à la transition vers un avenir énergétique plus durable et résilient.

Références

1. Andoni, M., Robu, V., Flynn, D., Abram, S., Geach, D., Jenkins, D., ... & Peacock, A. (2019). Blockchain technology in the energy sector: A systematic review of challenges and opportunities. *Renewable and Sustainable Energy Reviews*, 100, 143-174.

2. Buterin, V. (2014). Ethereum white paper: a next generation smart contract & decentralized application platform. Ethereum, 1-36.
3. Coelho, V. N., Weiss Cohen, M., Coelho, I. M., Liu, N., & Guimarães, F. G. (2017). Multi-agent systems applied for energy systems integration: State-of-the-art applications and trends in microgrids. *Applied energy*, 187, 820-832.
4. Mengelkamp, E., Gärttner, J., Rock, K., Kessler, S., Orsini, L., & Weinhardt, C. (2018). Designing microgrid energy markets: A case study: The Brooklyn Microgrid. *Applied Energy*, 210, 870-880.
5. Mihaylov, M., Jurado, S., Avellana, N., Van Moffaert, K., de Abril, I. M., & Nowé, A. (2014, May). NRGcoin: Virtual currency for trading of renewable energy in smart grids. In 11th International Conference on the European Energy Market (EEM14) (pp. 1-6). IEEE.
6. Pop, C., Cioara, T., Antal, M., Anghel, I., Salomie, I., & Bertoncini, M. (2018). Blockchain based decentralized management of demand response programs in smart energy grids. *Sensors*, 18(1), 162.
7. Ramchurn, S. D., Vytelingum, P., Rogers, A., & Jennings, N. R. (2012). Putting the 'smarts' into the smart grid: a grand challenge for artificial intelligence. *Communications of the ACM*, 55(4), 86-97.